

Python Args and Kwargs ----

args

You can access the *arguments* through the args variable:

```
# *args collects positional arguments into a tuple
def some_function(*args):
    print(f'Arguments passed: {args} as {type(args)}')

# Pass multiple arguments - they'll be collected into args tuple
some_function('arg1', 'arg2', 'arg3')
```

ex---

A student may have marks for **3 subjects**, another for **5 subjects**, and another for **8 subjects**.

```
def average_marks(*marks):
    avg = sum(marks) / len(marks)
    print("Average:", avg)
```

```
average_marks(80, 90, 70)
average_marks(85, 90, 95, 88, 92)
```

kwargs---

Keywords are accessed through the kwargs variable:

```
# **kwargs collects keyword arguments into a dictionary
def some_function(**kwargs):
    print(f'keywords: {kwargs} as {type(kwargs)}')

# Pass keyword arguments - they'll be collected into kwargs dict
some_function(key1='arg1', key2='arg2')
```

ex –

Every student may have different information.

```
def student_info(**details):  
    for key, value in details.items():  
        print(f"{key}: {value}")
```

```
student_info(  
    name="Shashank",  
    age=22,  
    course="MTech",  
    college="IIT Patna"  
)
```

Python Decorators---

A decorator in its simplest form is a function that takes another function as an argument and returns a wrapper. The following example shows the creation of a decorator and its usage.

Decorator: a function that takes another function and returns a wrapper

```
def your_decorator(func):  
    def wrapper():  
        # Do stuff before func...  
        print("Before func!")  
        func() # Call the original function  
        # Do stuff after func...  
        print("After func!")  
    return wrapper # Return the wrapper function
```

@your_decorator is syntactic sugar for: foo = your_decorator(foo)

@your_decorator

```
def foo():
```

```
print("Hello World!")
```

```
foo() # Calls wrapper, which calls foo with extra behavior
```

```
ex ----
```

Employee Attendance System

Before an employee starts work, attendance is marked.

```
def attendance(func):
```

```
    def wrapper():
```

```
        print("Attendance Marked")
```

```
        func()
```

```
    return wrapper
```

```
@attendance
```

```
def start_work():
```

```
    print("Employee Started Working")
```

```
start_work()
```

Decorators are used to add common functionality such as authentication, authorization, logging, execution-time measurement, caching, and input validation to multiple functions without modifying each function individually. This keeps the code reusable, clean, and easier to maintain.

Python OOP Basics---

Object-Oriented Programming (OOPs) is a programming paradigm that organizes code using objects and classes. It helps make code reusable, secure, modular, and easier to maintain.

The four main pillars of OOPs are:

1. Encapsulation
2. Abstraction
3. Inheritance
4. Polymorphism

1. Class

A class is a blueprint or template for creating objects.

Real-world example

Think of a Car blueprint.

The blueprint defines:

- Color
- Brand
- Model
- Speed

But it is not an actual car.

class Car:

 color = "Red"

 brand = "Toyota"

2. Object

An object is an instance of a class.

Real-world example

If the class is a Car blueprint, then an actual Toyota car parked outside is an object.

class Car:

 color = "Red"

car1 = Car()

print(car1.color)

3. Constructor (__init__)

A constructor is a special method that runs automatically when an object is created.

```
class Student:
```

```
    def __init__(self, name, age):  
        self.name = name  
        self.age = age
```

```
s1 = Student("Shashank", 22)
```

```
print(s1.name)
```

```
print(s1.age)
```

Encapsulation----

Definition

Encapsulation means binding data and methods together in a class and restricting direct access to sensitive data.

Real-world example



ATM Machine

You cannot directly change your account balance.

You must use:

- Deposit
- Withdraw
- Balance Enquiry

```
class Bank:
```

```
def __init__(self):  
    self.__balance = 10000
```

```
def show_balance(self):  
    return self.__balance
```

```
b = Bank()
```

```
print(b.show_balance())
```

Abstraction----

Definition

Abstraction means hiding implementation details and showing only the necessary functionality.

Real-world example

 **Driving a Car**

You use:

- **Steering**
- **Brake**
- **Accelerator**

```
from abc import ABC, abstractmethod
```

```
class Vehicle(ABC):
```

```
    @abstractmethod
```

```
    def start(self):
```

```
        pass
```

```
class Car(Vehicle):
```

```
    def start(self):
```

```
print("Car Started")
```

```
c = Car()
```

```
c.start()
```

Inheritance-----

Definition

Inheritance allows one class to acquire the properties and methods of another class.

Real-world example

Father → Son

The son inherits many characteristics from the father.

class Animal:

```
    def eat(self):  
        print("Eating")
```

class Dog(Animal):

```
    pass
```

```
d = Dog()
```

```
d.eat()
```

Types of Inheritance

Single Inheritance

Animal

|

Dog

Multiple Inheritance

Father Mother

|

Child

class A:

pass

class B:

pass

class C(A, B):

pass

Multilevel Inheritance---

Grandfather

|

Father

|

Son

Hierarchical Inheritance-

Animal

/ \

Dog Cat

Hybrid Inheritance

A combination of multiple inheritance types.

Polymorphism-----

Definition

Polymorphism means one interface, many forms.

The same method behaves differently for different objects.

Real-world example

Calling

- Call your friend
- Call your office
- Call customer care

The action is the same ("call"), but the result depends on whom you call.

Method Overriding

```
class Animal:
```

```
    def sound(self):  
        print("Animal Sound")
```

```
class Dog(Animal):
```

```
    def sound(self):  
        print("Bark")
```

```
d = Dog()
```

```
d.sound()
```

Method Overloading---

Python does not support traditional method overloading.

Instead, you usually use default arguments or *args

```
class Demo:
```

```
    def add(self, a, b=0, c=0):
```

```
print(a + b + c)
```

```
d = Demo()
```

```
d.add(10)
```

```
d.add(10, 20)
```

```
d.add(10, 20, 30)
```

Method Overriding

A child class provides its own implementation of a method defined in the parent class.

```
class Animal:
```

```
    def sound(self):  
        print("Animal")
```

```
class Dog(Animal):
```

```
    def sound(self):  
        print("Bark")
```

super() Keyword---

super() is used to call the parent class constructor or methods.

```
class Animal:
```

```
    def __init__(self):  
        print("Animal Constructor")
```

```
class Dog(Animal):
```

```
    def __init__(self):
```

```
        super().__init__()
```

```
        print("Dog Constructor")
```

```
Dog()
```

Difference between Class and Object

Class	Object
Blueprint	Real instance
Logical entity	Physical entity
Defines properties	Uses properties

Difference between Encapsulation and Abstraction

Encapsulation	Abstraction
Protects data	Hides implementation
Uses private members	Uses abstract classes/interfaces
Focuses on data security	Focuses on simplicity

Difference between Overloading and Overriding

Overloading	Overriding
Same method, different parameters	Child class redefines parent method
Not directly supported in Python	Fully supported

1. Create a Student class with attributes name, age, and course. Create two objects and display their details.
2. Create an Employee class with id, name, and salary. Add a method to display employee details.
3. Create a Car class with brand, model, and year. Add a method to start the engine.
4. Create a Rectangle class and write methods to calculate the area and perimeter.
5. Create a BankAccount class with methods to deposit, withdraw, and check balance.
6. Create a Book class with title, author, and price. Display information for three books.
7. Create a Mobile class with methods to make a call, send a message, and open the camera.
8. Create a Laptop class with company, RAM, processor, and price. Display all details.
9. Create a Movie class with methods to book and cancel tickets.
10. Create an ATM class with methods for deposit, withdrawal, and balance enquiry.
11. Create a Student class where marks is a private variable. Add methods to set and get marks.
12. Create a BankAccount class with a private balance variable. Allow balance updates only through methods.
13. Create a PasswordManager class with a private password. Add methods to set and verify the password.
14. Create a Patient class with private medical records. Provide methods to update and view records.
15. Create an Animal class and inherit it into a Dog class. Demonstrate inheritance.
16. Create a Vehicle class and inherit it into Car and Bike classes. Add specific methods to each child class.
17. Create a Person class and inherit it into Teacher and Student classes. Display all details using inherited methods.
18. Create classes Circle, Rectangle, and Triangle, each with an area() method. Demonstrate polymorphism.
19. Create an abstract class Payment with an abstract method pay(). Implement subclasses UPI, CreditCard, and NetBanking.
20. Design a Library Management System using OOP with classes such as Book, Member, Librarian, and Library. Include features like adding books, issuing books, returning books, and viewing available books.

lunetron web services